



UAC



CEA-SMIA

THE ABDUS SALAM
INTERNATIONAL CENTER FOR
THEORETICAL PHYSICS
(ICTP)



Dangbo (BENIN)

UNIVERSITE D'ABOMEY-CALAVI (UAC) INSTITUT DE MATHÉMATIQUES ET DE SCIENCES PHYSIQUE (IMSP)

*Centre d'Excellence Africain en Sciences Mathématiques,
Informatiques et Applications (CEA-SMIA)*

PROJET SCIENTIFIQUE
pour l'obtention de
LICENCE SPECIALE DES CLASSES PREPARATOIRES
Option : Mathématiques

THEME

MÉTHODES (ALGORITHMES) DE DESCENTE ET
APPLICATIONS À LA CLASSIFICATION
SUPERVISÉE

Rédigé par :

INADJRO Rock

rockinadjro@gmail.com

&

WANTI Denis

deniswanti0@gmail.com

Superviseur

Prof. Guy DEGLA

gdegla@imsp-uac.org

Encadreur

Dr. Japhet ODJOUMANI

japhet.odjoumani@imsp-uac.org

Membres de Jury

Président : Prof. Léonard TODJIHOUNDE

Membre : Prof. Guy DEGLA

Membre : Dr. Japhet ODJOUMANI

Présentation

Dangbo, le 29 / 07 / 2024

Dédicaces

À

- ★ Ma tante *Rose LINKPEHOUN*
- ★ Ma mère *Margueritte LINKPEHOUN*.

Rock INADJRO

À

- ★ Mon oncle Hippolyte N'DAH
- ★ Ma sœur *Lucrèce WANTI*.
- ★ Mon père *Séraphin WANTI* .

Dénis WANTI

Remerciements

Nous tenons tout d'abord à remercier le Professeur Guy DEGLA et le Docteur Japhet ODJOUMANI, d'avoir accepté de nous encadrer, pour les compétences mises à notre disposition, leurs bonnes directives, et pour le temps qu'ils ont consacré à la réalisation de ce travail, qu'ils trouvent ici l'expression de notre profonde gratitude.

Nous tenons également à remercier tout le personnel administratif de l'IMSP pour le soutien qu'il nous a toujours apporté.

Nous ne passerons cette occasion sans remercier tous les enseignants des Classes Préparatoires de l'IMSP pour nous avoir inculqué des bases solides dans leurs disciplines respectives.

Finalement, nous ne pouvons oublier de remercier profondément les êtres les plus chers, nos parents, pour leur soutien, leurs encouragements et la confiance qu'ils nous accordaient depuis toujours, ainsi que nos amis pour tous les bons moments passés ensemble. ENCORE MERCI !

Introduction

Dans un monde de plus en plus axé sur les données, les méthodes de classification supervisée jouent un rôle crucial dans divers domaines, allant de la reconnaissance d'images à la détection de spams en passant par la prédiction de maladies. La classification supervisée permet de classer des données nouvelles en fonction de connaissances préalablement acquises à partir d'un ensemble d'entraînement étiqueté. Au cœur de ces algorithmes de classification se trouvent souvent des problèmes d'optimisation, dont la résolution est rendue possible grâce à des méthodes de descente. Les méthodes de descente ont leurs mérites propres et sont des algorithmes itératifs qui permettent de minimiser une fonction de coût en ajustant progressivement les paramètres du modèle. Ces techniques sont essentielles pour entraîner des modèles de machine learning efficaces, notamment dans le cadre de la classification supervisée, où la précision des modèles dépend de la capacité à optimiser les paramètres de manière efficace et fiable.

Le présent mémoire vise à explorer diverses méthodes de descente et puis à les appliquer dans la résolution des modèles d'optimisation permettant la classification supervisée. Plus précisément, les objectifs sont les suivants : présenter et expliquer les fondements théoriques des principales méthodes de descente, incluant la descente de gradient standard, ainsi que ses variantes et extensions telles que la descente de gradient stochastique. Ensuite, examiner les performances de ces méthodes à travers une application dans le cadre de la régression logistique, afin de mettre en évidence leurs avantages et inconvénients respectifs. En démontrant l'application pratique de ces algorithmes dans des modèles de classification supervisée, nous montrerons comment ils peuvent être utilisés pour optimiser des modèles de régression logistique, les machines à vecteurs de support (SVM), et les réseaux de neurones.

Ce mémoire se veut à la fois un guide pratique pour les chercheurs et les praticiens de l'apprentissage supervisée (machine learning), ainsi qu'une contribution académique enrichissant la littérature existante sur l'optimisation des modèles de classification supervisée. En éclairant les tenants et aboutissants des différentes méthodes de descente, nous espérons fournir des outils et des connaissances permettant de développer des modèles plus précis et efficaces, et ainsi de répondre aux défis croissants posés par la classification des données dans divers contextes applicatifs.

Table des matières

Dédicaces	ii
Remerciements	iii
Introduction	iv
1 Généralités sur les méthodes de descente	2
1.1 Rappels et notions élémentaires	2
1.1.1 Dérivée directionnelle :	3
1.1.2 Convexité et optimisation.	4
1.1.3 Résultat d'existence, cas d'unicité :	5
1.1.4 Condition d'optimalité	5
1.2 Direction de descente	7
1.3 Recherche linéaire : méthode de recherche du pas de descente	8
1.4 Algorithme de descente	9
1.5 Convergence et vitesse de convergence	10
2 Méthodes de descente	12
2.1 Méthodes du gradient	12
2.1.1 Méthode du gradient à pas fixe	15
2.1.2 Méthode du gradient à pas optimal	16
2.1.3 Méthode du gradient conjugué	17
2.1.4 Méthode du gradient stochastique	19
2.2 Méthode de Newton	20
3 Applications : Classification supervisée	21
3.1 Définition	21
3.2 Exemples	21
3.3 Algorithmes de classification	21
3.4 Quelques problèmes de classification supervisée	24
3.4.1 Classification d'images	24
3.4.2 Classification de texte	24
3.5 Une illustration	24
3.6 Analyse des résultats	27
Conclusion	30

Généralités sur les méthodes de descente

1.1 Rappels et notions élémentaires

Commençons ce paragraphe par quelques rappels et notions élémentaires.

1. Pour tout $n \in \mathbb{N}^*$, $\mathbb{R}^n$ désigne l'espace euclidien $\mathbb{R} \times \dots \times \mathbb{R}$ (produit n fois). En général un vecteur $x \in \mathbb{R}^n$ sera noté $x = (x_1, x_2 \dots x_n)^T$ (vecteur colonne).
2. On considère dans cette partie une fonction f définie de $\mathbb{R}^n$ vers $\mathbb{R}$.
On dit que f est continue en $x \in \mathbb{R}^n$ si $f(x^{(k)}) \rightarrow f(x)$ pour toute suite $x^{(k)} \subset \mathbb{R}^n$ telle que $x^{(k)} \rightarrow x$. On dit que f est continue sur $\mathbb{R}^n$ si f est continue en tout point $x \in \mathbb{R}^n$.

Définition 1.1. Soit $f : \Omega \subset \mathbb{R}^n \rightarrow \mathbb{R}$. Soit a un point intérieur de Ω . On dit que f est différentiable en a s'il existe une application linéaire (et continue) $L \in \mathbb{R}^n \rightarrow \mathbb{R}$ telle que

$$\lim_{h \rightarrow 0} \frac{f(a+h) - f(a) - L(h)}{\|h\|} = 0 \quad (1.1)$$

$$\text{En posant } \varepsilon(h) = \begin{cases} \frac{f(a+h) - f(a) - L(h)}{\|h\|} & \text{si } h \neq 0 \\ 0 & \text{sinon} \end{cases} \quad \text{on a}$$

$$(1.1) \iff f(a+h) = f(a) + L(h) + \|h\|\varepsilon(h)$$

avec

$$\lim_{h \rightarrow 0} \varepsilon(h) = 0$$

ce qu'on écrit encore sous la forme $f(a+h) = f(a) + L(h) + o(h)$.

Proposition 1.1. Si f est différentiable en a , on montre que $L(h) = \nabla f(a) \cdot h$ où $\nabla f(a) = \left(\frac{\partial f}{\partial x_i}(a) \right)_{1 \leq i \leq n}$.

Remarque 1.1. f est différentiable en a si elle possède des dérivées partielles d'ordre 1 en a et de plus $\lim_{h \rightarrow 0} \frac{f(a+h) - f(a) - L(h)}{\|h\|} = 0$.

Théorème 1.1. Si Ω est un ouvert non vide et f possède des dérivées partielles continues sur Ω , alors f est différentiable sur Ω et de plus l'application $x \mapsto \nabla f(x)$ est continue. On dit que f est de classe $\mathcal{C}^1$.

1.1.1 Dérivée directionnelle :

Définition 1.2. Soit $f : \mathbb{R}^n \rightarrow \mathbb{R}$ une application continue. Soit $x \in \mathbb{R}^n$ et $d \in \mathbb{R}^n$. La dérivée directionnelle de f en x dans la direction d si elle existe est définie par :

$$df(x; d) := \lim_{t \rightarrow 0^+} \frac{f(x + td) - f(x)}{t}.$$

Proposition 1.2. Si f est différentiable en un point $x \in \mathbb{R}^n$, alors pour tout $d \neq 0$, f admet une dérivée dans la direction d en x et :

$$df(x; d) = Df(x)(d) = \nabla f(x)^T d.$$

On rappelle que la réciproque est fausse : La dérivée directionnelle de f en x selon tout vecteur d n'implique pas nécessairement la différentiabilité de f en x .

Exemple 1.1. La fonction f définie sur $\mathbb{R}^2 \setminus \{0\} \times \mathbb{R}$ par $f(x, y) = \frac{y^2}{x}$ et telle que $f(0, y) = 0$ pour tout y admet des dérivées directionnelles en toute direction en $(0, 0)$, mais n'est pas continue en $(0, 0)$. En effet, pour tout vecteur $\vec{u} = (\alpha, \beta)$ et tout $t \in \mathbb{R}^*$, on a $f(t\vec{u}) = t\beta^2/\alpha$ si $\alpha \neq 0$, et $f(t\vec{u}) = 0$ si $\alpha = 0$. Dans tous les cas $df((0, 0); \vec{u}) = 0$.

Définition 1.3. On dit qu'un point $x^* \in \mathbb{R}^n$ est un point critique (ou stationnaire) pour la fonction f si $\nabla f(x^*) = 0$.

Position du problème.

Rappelons qu'une contrainte est une condition logique (égalité ou inégalité) que doit satisfaire la solution d'un problème d'optimisation.

Un problème d'optimisation sans aucune condition est appelé un problème sans contrainte.

On considère ici le cas particulier où l'ensemble de contraintes est défini sur l'espace $\mathbb{R}^n$ tout entier. On va considérer le problème de minimisation suivant :

$$\min_{x \in \mathbb{R}^n} f(x)$$

où $f : \mathbb{R}^n \rightarrow \mathbb{R}$ est une fonctionnelle donnée.

C'est un problème de minimisation sans contraintes. On pourra donc écrire ce problème sous forme :

$$\begin{cases} \text{Trouver } x^* \in \mathbb{R}^n \text{ tel que} \\ f(x^*) \leq f(x), \quad \forall x \in \mathbb{R}^n. \end{cases}$$

On supposera que x^* existe (éventuellement qu'il est unique) et on se propose de trouver une approximation numérique de x^* , en construisant une suite d'itérés $(x^{(k)})_{k \in \mathbb{N}} \subset \mathbb{R}^n$ telle que $x^{(k)} \rightarrow x^*$ pour $k \rightarrow +\infty$.

Cette suite d'itérés sera construite par les méthodes numériques de minimisation que nous verrons plus loin.

Résoudre un problème d'optimisation sans contraintes revient à chercher des points de minimum local (ou global), au sens de la définition suivante.

Définition 1.4. (Minimum local/ Minimum global)

Soit $f : U \subset \mathbb{R}^n \rightarrow \mathbb{R}$ une fonctionnelle donnée.

- $x^* \in \mathbb{R}^n$ est un point de minimum local de f sur U si :
 $x^* \in U$ et $\exists r > 0$ tel que $\forall x \in U \cap B(x^*, r)$, $f(x^*) \leq f(x)$.
On dit alors que $f(x^*)$ est un minimum local de f sur U .

- $x^* \in \mathbb{R}^n$ est un point de minimum global de f sur U si et seulement si :
 $x^* \in U$ et $\forall x \in U, \quad f(x^*) \leq f(x)$.
On dit alors que $f(x^*)$ est un minimum global de f sur U .

Les notions de maximum local et global sont définies de façon tout à fait similaire. En fait, on peut facilement démontrer que les problèmes sans contraintes : $\min_{x \in \mathbb{R}^n} f(x)$ et $\max_{x \in \mathbb{R}^n} (-f(x))$ sont équivalents dans le sens où ils ont même ensemble de solutions. Ainsi la recherche d'un maximum peut se ramener à la recherche d'un minimum, nous porterons donc une attention particulière à la recherche d'un minimum.

1.1.2 Convexité et optimisation.

La convexité joue un rôle extrêmement important en optimisation numérique. Les problèmes dont les données sont convexes, constituent une classe importante, car ils sont fréquemment rencontrés dans les applications et à la base de nombreuses méthodes développées pour des problèmes plus généraux.

Définition 1.5. (*ensemble convexe*)

Un ensemble C est dit convexe si, pour tous points x et y de C , le segment $[x, y]$ est inclus dans C , c-à-d $\forall t \in [0, 1], tx + (1 - t)y \in C$.

Exemple 1.2. $\mathbb{R}^n$ est un ensemble convexe.

Définition 1.6. (*fonction convexe*)

Une fonction f définie sur un ensemble convexe C est dite convexe si et seulement si :

$$\forall x, y \in C, \quad \forall t \in [0, 1], \quad f(tx + (1 - t)y) \leq tf(x) + (1 - t)f(y).$$

La fonction est dite strictement convexe si et seulement si :

$$\forall x, y \in C, \quad x \neq y, \quad \forall t \in]0, 1[, \quad f(tx + (1 - t)y) < tf(x) + (1 - t)f(y).$$

Lorsqu'une fonction convexe est dérivable, la caractérisation suivante sera utile.

Proposition 1.3. Soit f une fonction différentiable définie sur un convexe C de $\mathbb{R}^n$, alors f est convexe si et seulement si :

$$\forall x, y \in C, \quad \nabla f(x)(y - x) \leq f(y) - f(x).$$

Remarque 1.2. On a une caractérisation similaire pour les fonctions strictement convexes : remplacer convexe par strictement convexe, et l'inégalité large par une inégalité stricte dans la proposition précédente.

Proposition 1.4. Soit f une fonction convexe définie sur un ensemble convexe C . Alors ;

- Tout minimum local de f sur C est un minimum global.
- Si f est strictement convexe, il y a au plus un minimum global (unicité du minimum global).

Démonstration.

- Soit $x_0 \in C$ un minimum local, i.e. il existe $r > 0$ tel que :

$$\forall x \in B(x_0, r) \cap C, \quad f(x) \geq f(x_0).$$

Supposons qu'il existe $y \in C$ tel que $f(y) < f(x_0)$

Soit $t \in]0, 1[$ et posons $y_t = ty + (1 - t)x_0$. Puisque $\|y_t - x_0\| = t\|y - x_0\|$, si t est suffisamment petit, y_t sera au voisinage de x_0 . Donc $f(y_t) \geq f(x_0)$.

Comme f est convexe sur C on a

$$f(x_0) \leq f(y_t) \leq tf(y) + (1 - t)f(x_0) \implies f(x_0) \leq f(y)$$

ce qui contredit l'hypothèse donc x_0 est un minimum global.

- Soit f strictement convexe. On suppose que f admet deux minimum globaux distincts x, y .

Soit alors $t \in]0, 1[$ et $z = tx + (1 - t)y$.

On aboutit à la contradiction suivante : $f(z) < tf(x) + (1 - t)f(y) = \min_C f$.

D'où l'unicité du point de minimum global.

□

1.1.3 Résultat d'existence, cas d'unicité :

La plupart des théorèmes d'existence de minimum sont des variantes du théorème classique suivant : une fonction continue sur un compact admet un minimum.

Commençons par le théorème suivant :

Théorème 1.2. *Soit f une fonction continue sur un sous-ensemble C fermé de $\mathbb{R}^n$. On suppose que :*

- Ou bien C est borné,
- Ou bien C est non borné et $\lim_{\|x\| \rightarrow +\infty} f(x) = +\infty$ (on dit que f est coercive),

Alors f possède un minimum sur C .

Le résultat d'unicité du point de minimum global est prouvé précédemment, sous la condition de convexité stricte de f .

1.1.4 Condition d'optimalité

Nous supposons dans tout ce paragraphe que f est une ou deux fois différentiable. On notera x^* un minimum (local) de f . Ce qui suit reste valable dans le cas où le minimum x^* se trouve à l'intérieur de l'ensemble des contraintes. Nous donnons les conditions nécessaires de minimum, puis celles suffisantes.

Théorème 1.3. *(conditions nécessaires.)*

Les deux conditions nécessaires sont les suivantes.

- Condition au premier ordre : si f est différentiable en x^* , on a :

$$\nabla f(x^*) = 0,$$

- Condition au second ordre : si f est deux fois différentiable au point x^* , alors la forme quadratique $D^2f(x^*)$ est semi positive c-a-d

$$\langle D^2f(x^*)y, y \rangle \geq 0, \forall y \in \mathbb{R}^n$$

où $D^2f(x^*)$ est la matrice Hessienne, définie par les coefficients : $\frac{\partial^2 f}{\partial x_i \partial x_j}(x^*)$.

Démonstration.

i) On considère un développement limité à l'ordre un en x^* :

$$f(x^* + h) = f(x^*) + \nabla f(x^*)h + \|h\|E(x; h), \quad h \in \mathbb{R}^n.$$

Soit $t \geq 0$. On choisit $h = -t\nabla f(x^*)$ et on pose $\alpha = \|\nabla f(x^*)\|$. Ce qui donne :

$$f(x^* + h) = f(x^*) - t\alpha^2 + t\alpha E(x^*, h).$$

Supposons que $\alpha > 0$. Puisque $E(x^*, h) \xrightarrow{t \rightarrow 0} 0$ on vérifie alors qu'il existe $t^* > 0$ tel que : $\forall t \leq t^* : f(x^* + h) < f(x^*)$. Ceci indique que x^* n'est pas un minimum. Ainsi il est nécessaire que $\alpha = 0$.

ii) Supposons qu'il existe $y \in \mathbb{R}^n, y \neq 0$ tel que $\alpha := \langle D^2 f(x^*)y, y \rangle < 0$. On considère un développement limité à l'ordre deux en x^* :

$$f(x^* + ty) = f(x^*) + \frac{1}{2}t^2\alpha + t^2\alpha^2 E_2(x^*, ty).$$

Puisque $E_2(x^*, ty) \xrightarrow{t \rightarrow 0} 0$, on vérifie qu'il existe $t > 0$ tel que $\forall t \in [0; t^*] : f(x^* + ty) < f(x^*)$. Ceci indique que x^* n'est pas un minimum. Ainsi il est donc nécessaire que $D^2 f(x^*)$ soit semi définie positive. Ce qui achève la preuve de ce théorème.

Nous énonçons à présent des conditions nécessaires et suffisantes pour qu'un point x^* soit un minimum, dans le cas où f est suffisamment régulière.

□

Théorème 1.4. (Conditions nécessaires et suffisantes).

Soit f une fonction de classe $\mathcal{C}^1$ définie sur $\mathbb{R}^n$. On suppose que $\nabla f(x^*) = 0$ et que f deux fois différentiable en x^* .

Alors, x^* est un minimum (local) de f si et seulement si l'une des deux conditions suivantes est vérifiée :

i) $D^2 f(x^*)$ est définie positive.

ii) $\exists r > 0$ tel que f est deux fois différentiable sur $B(x^*, r)$ et la forme quadratique $D^2 f(x)$ est semi définie positive pour tout $x \in B(x^*, r)$.

Démonstration.

- la condition nécessaire au premier ordre et au deuxième ordre en x^* est vérifiée dans les deux cas i) et ii).
- On vérifie que i) ou ii) est une condition suffisante.

Pour ii) : on considère la formule de Taylor-Mac-Laurin en x^* à l'ordre deux, pour tout $h \in B(0, r)$ il existe $\lambda = \lambda(h) \in [0, 1]$, donnée par :

$$f(x^* + h) = f(x^*) + \frac{1}{2} \langle D^2 f(x^* + \lambda h)h, h \rangle \geq f(x^*),$$

ce qui montre bien que x^* est un minimum.

Ce qui achève la preuve du théorème.

□

Dans le cas où f est convexe, la condition suffisante s'exprime beaucoup plus facilement. En effet, nous avons la proposition suivante :

Proposition 1.5. Soit f une fonction convexe de classe $\mathcal{C}^1$, définie sur $\mathbb{R}^n$ et x^* un point de $\mathbb{R}^n$. Alors, x^* est un minimum (global) de f si et seulement si :

$$\nabla f(x^*) = 0.$$

Démonstration. Il suffit de montrer que la condition est suffisante. Soit $y \in \mathbb{R}^n$ quelconque. D'après la proposition précédente nous avons :

$$f(y) - f(x^*) \geq \nabla f(x^*) (y - x^*) = 0$$

Ce qui montre bien que x^* est un minimum. $\square$

1.2 Direction de descente

Définition 1.7. Soit $f : \mathbb{R}^n \rightarrow \mathbb{R}$ et $x \in \mathbb{R}^n$. Un vecteur $d \in \mathbb{R}^n$ est une direction de descente pour f à partir du point x si et seulement s'il existe un réel $\delta > 0$, tel que $\forall t \in]0, \delta]$ on a :

$$f(x + td) < f(x).$$

Proposition 1.6. Soit $f : \mathbb{R}^n \rightarrow \mathbb{R}$ une fonction différentiable au point $x \in \mathbb{R}^n$ et $d \in \mathbb{R}^n$ telle que

$$(\nabla f(x))^T \cdot d < 0.$$

Alors d est une direction de descente au point x .

Démonstration. f est différentiable au point x . Donc :

$$f(x + \rho d) = f(x) + \rho \nabla f(x)^T \cdot d + \rho \|d\| o(x, \rho d), \text{ avec } o(\rho d) \xrightarrow{\rho \rightarrow 0} 0$$

Ceci implique que

$$\lim_{\rho \rightarrow 0} \frac{f(x + \rho d) - f(x)}{\rho} = (\nabla f(x))^T \cdot d < 0$$

La limite étant strictement négative, alors il existe un voisinage de zéro $v(0) =]-\alpha, +\alpha[$ tel que :

$$\frac{f(x + \rho d) - f(x)}{\rho} < 0, \forall \rho \in]-\alpha, +\alpha[$$

La relation précédente est particulièrement vraie pour tout $\rho \in]0, +\alpha[$. On obtient le résultat cherché en multipliant cette relation par $\rho > 0$.

C'est à dire :

$$f(x + \rho d) < f(x) \text{ pour tout } \rho \in]0, +\alpha[$$

L'ensemble des directions de descente de f en x est donné par :

$$\{d \in \mathbb{R}^n : \nabla f(x)^T \cdot d < 0\}$$

$\square$

Théorème 1.5. Soit $f : \mathbb{R}^n \rightarrow \mathbb{R}$ une fonction différentiable.

Soit $x \in \mathbb{R}^n$, alors pour toute direction de norme constante égale à $\|d\| = \|\nabla f(x)\|$ on a

$$-\nabla f(x) \cdot (\nabla f(x))^T \leq \nabla f(x) \cdot d^T$$

La direction $d^* = -\nabla f(x)$ est appelé direction de plus forte pente.

Démonstration.

Soit $d \in \mathbb{R}^n$ une direction quelconque de norme : $\|d\| = \|\nabla f(x)\|$.

D'après l'inégalité de Cauchy Schwartz et le fait que $\|d\| = \|\nabla f(x)\|$

$$\nabla f(x) \cdot d^T \leq \|d\| \|\nabla f(x)\| \leq \|\nabla f(x)\|^2 = \nabla f(x) \cdot (\nabla f(x))^T$$

On en déduit que $-\nabla f(x) \cdot (\nabla f(x))^T \leq \nabla f(x) \cdot d^T$

Si l'opposé du gradient correspond à la plus forte descente, le gradient correspond, lui, à la plus forte montée. $\square$

Corollaire 1.1. *Le vecteur $\nabla f(x)$ est appelé direction de plus forte pente de f au point $x \in \mathbb{R}^n$. On remarquera que si x^* est un point minimum local de f , alors il n'existe aucune direction de descente pour f au point x^* .*

Cette direction étant choisie, nous sommes plus ou moins ramené à un problème unidimensionnel pour la détermination de $\rho^{(k)}$.

Pour ces raisons, commençons par analyser ce qui se passe dans le cas de dimension 1.

1.3 Recherche linéaire : méthode de recherche du pas de descente

En optimisation mathématique, la recherche linéaire est l'une des deux approches classiques permettant de forcer et d'accélérer la convergence des algorithmes de calcul d'un minimum x^* d'une fonction $f : \mathbb{R}^n \rightarrow \mathbb{R}$, lorsque le premier itéré est éloigné d'un tel minimum.

Dans cette partie nous nous intéressons à la recherche linéaire du pas de descente d'un point minimum x^* .

Soit $q(\rho)$ la fonction coût que l'on cherche à minimiser.

On pourra prendre par exemple : $q(\rho) = f(x^{(k)} + \rho d^{(k)})$ afin d'appliquer les idées au cas de la méthode de descente.

La recherche du pas n'est qu'une étape d'un algorithme plus complexe pour minimiser f .

Si nous considérons $q(\rho) = f(x + \rho d)$, avec $\rho > 0$, nous avons :

$$q'(\rho) = \nabla f(x + \rho d) \cdot d$$

Par conséquent, puisque d est une direction de descente, $\nabla f(x) \cdot d < 0$, il s'ensuit que si nous prenons ρ un peu à droite de 0, on est sûr de faire décroître q . Toutefois, il faut faire attention car deux éléments contradictoires sont à prendre en compte.

- Si ρ est trop grand, on risque de ne pas faire décroître la fonction q où son comportement peut être oscillant,
- Si ρ est trop petit, l'algorithme n'avancera pas assez vite.

Il existe des règles classiques pour parvenir à cette fin.

a) Règle d'Armijo (1966)

La règle d'Armijo se base sur le choix d'un paramètre $0 < m < 1$ et détermine une valeur approchée de ρ sous la condition :

$$q(\rho) \leq q(0) + m\rho q'(0).$$

Le risque de cette méthode est de favoriser les valeurs trop petites, aussi, elle est rarement utilisée seule.

b) Règle de Goldstein (1967) :

On choisit deux paramètres m_1 et m_2 tels que $0 < m_1 < m_2 < 1$ et on cherche une valeur ρ qui vérifie :

$$\begin{cases} q(\rho) \leq q(0) + m_1 \rho q'(0) \\ q(\rho) \geq q(0) + m_2 \rho q'(0) \end{cases}$$

c) Règle de Wolfe (1969) :

On choisit deux paramètres m_1 et m_2 , avec $0 < m_1 < m_2 < 1$ (par exemple $m_1 = 0.1$ et $m_2 = 0.7$), et on recherche ρ qui vérifie :

$$\begin{cases} q(\rho) \leq q(0) + m_1 \rho q'(0) \\ q(\rho) \geq m_2 q'(0) \end{cases}$$

L'algorithme associé à la règle de Wolfe est alors donné par la table ci-dessous, celui pour la méthode de Goldstein étant similaire.

Soit f la fonction à minimiser.

A l'itération k , nous avons $x = x^{(k)}$ et $d = d^{(k)}$, et on veut calculer : $x^{(k+1)} = x^{(k)} + \rho d^{(k)}$ pour une valeur ρ à déterminer.

1. Partir de $k = 0$; Choisir un point $x^{(0)}$ de $\mathbb{R}^n$ et m_1 et m_2 tel que : $0 < m_1 < m_2 < 1$;
Poser $\rho = 1$; $\rho_+ = 0$; $\rho_- = 0$
2. Tant que $(k \leq k_{\max})$
 - Si la première condition de Wolfe n'est pas vérifiée : $q(\rho) \leq q(0) + m_1 \rho q'(0)$, alors le pas est trop long
Poser : $\rho_+^{(k)} = \rho^{(k)}$; Et $\rho^{(k+1)} = \frac{\rho_- + \rho_+}{2}$;
Poser $k = k + 1$
 - Sinon si la deuxième condition de Wolfe n'est pas vérifiée : $q(\rho) \geq m_2 q'(0)$, alors le pas est trop court
Poser : $\rho_-^{(k)} = \rho^{(k)}$;
Si $\rho_+ = 0$ on a : $\rho^{(k+1)} = 2\rho^{(k)}$;
Sinon, on a : $\rho^{(k+1)} = \frac{\rho_- + \rho_+}{2}$;
Poser $k = k + 1$;
3. Retourner $\rho^{(k)}$; Fin.

1.4 Algorithme de descente

Partant d'un point $x^{(0)}$ arbitrairement choisi, un algorithme de descente va chercher à générer une suite d'itérés $(x^{(k)})_{k \in \mathbb{N}}$ telle que :

$$\forall k \in \mathbb{N}, f(x^{(k+1)}) \leq f(x^{(k)}).$$

D'après la caractérisation de la descente, il s'agit donc à chaque itération k , de trouver un point $x^{(k+1)}$ dans une direction d vérifiant : $\nabla f(x^{(k)})^T d < 0$.

Le schéma général d'un algorithme de descente est le suivant :

Entrée : $f : \mathbb{R}^n \rightarrow \mathbb{R}$ supposé au moins différentiable, $x^{(0)}$ un point initial arbitrairement choisi.

Sortie : une approximation de la solution du problème : $\min_{x \in \mathbb{R}^n} f(x)$.

1. Initialisation $k = 0$.
2. Tant que test de convergence non satisfait.
 - a) Trouver une direction de descente $d^{(k)}$ tel que : $\nabla f(x^{(k)})^T d^{(k)} < 0$.
 - b) Recherche linéaire : Choisir un pas $\rho^{(k)} > 0$ à faire dans cette direction tel que :

$$f(x^{(k)} + \rho^{(k)} d^{(k)}) \leq f(x^{(k)})$$

c) Mise à jour :

$$x^{(k+1)} = x^{(k)} + \rho^{(k)} d^{(k)}$$

Poser $\leftarrow k = k + 1$;

3. Retourner $x^{(k)}$.

Test de convergence

Soit x^* un point de minimum local du critère f à optimiser.

Supposons que l'on choisisse comme test d'arrêt dans l'algorithme de descente, le critère idéal : $x^{(k)} = x^*$.

En pratique, un test d'arrêt devra être choisi pour garantir que l'algorithme s'arrête toujours après un nombre fini d'itérations et que le dernier point calculé soit suffisamment proche de x^* .

Soit $\varepsilon > 0$ la précision demandée, on test si $\|\nabla f(x_k)\| \leq \varepsilon$.

Dans ce cas l'algorithme s'arrête et fournit l'itéré courant $x^{(k)}$ comme solution.

En pratique, le test d'optimalité n'est pas toujours satisfait et on devra faire appel à d'autres critères (fondés sur l'expérience numérique) ;

- Stagnation de solution : $\|x^{(k+1)} - x^{(k)}\| < \varepsilon \|x^{(k)}\|$.
- Stagnation de la valeur courante : $\|f(x^{(k+1)}) - f(x^{(k)})\| < \varepsilon \|f(x^{(k)})\|$.
- Nombre d'itérations dépassant un seuil fixé à l'avance : $k > k_{\max}$.

Et généralement une combinaison de ces critères :

Critère d'arrêt = Test d'optimalité satisfait

OU (Stagnation de la valeur courante et stagnation de la solution) OU Nombre d'itérations maximum autorisé dépassé.

1.5 Convergence et vitesse de convergence

Étudier la convergence d'un algorithme, c'est étudier la convergence de la suite des itérés $(x^{(k)})_{k \in \mathbb{N}}$ générés par l'algorithme.

Un algorithme de descente selon le modèle précédent, est dit convergent si la suite de ses itérés $(x^{(k)})_{k \in \mathbb{N}}$ converge vers un point limite x^* , solution du problème :

$$\min_{x \in \mathbb{R}^n} f(x)$$

De plus, la convergence est dite locale si elle n'a lieu que pour des points initiaux $x^{(0)}$ dans un voisinage de x^* .

Sinon elle est dite globale.

En pratique, le but d'un algorithme d'optimisation est de trouver un point critique (i.e. un point vérifiant la condition d'optimalité du premier ordre : $\nabla f(x^*) = 0$).

On introduit alors la notion de convergence d'un algorithme d'optimisation.

Définition 1.8. Soit un algorithme itératif qui génère une suite $(x^{(k)})_{k \in \mathbb{N}}$ dans $\mathbb{R}^n$ afin de résoudre le problème :

$$\min_{x \in \mathbb{R}^2} f(x),$$

où $f : \mathbb{R}^n \rightarrow \mathbb{R}$ est une application de classe C^1 .

L'algorithme est dit globalement convergent si quel que soit le point initial $x^{(0)} \in \mathbb{R}^n$,
 $\lim_{k \rightarrow +\infty} \|\nabla f(x^{(k)})\| = 0$.

Cette propriété garantit que le critère d'arrêt $\|\nabla f(x^{(k)})\| \leq \varepsilon$ sera satisfait à partir d'un certain rang quelque soit la précision $\varepsilon > 0$, demandée.

Il est bien entendu très important de garantir la convergence d'un algorithme sous certaines hypothèses, mais la vitesse de convergence et la complexité sont également des facteurs à prendre en compte lors de la conception ou de l'utilisation d'un algorithme ; en effet, on a tout intérêt à ce que la méthode choisie soit à la fois rapide, précise et stable.

Pour cela, on introduit les notions de vitesse (ou taux) de convergence qui mesurent l'évolution de l'erreur commise : $\|x^{(k)} - x^*\|$.

Définition 1.9. Soit $(x^{(k)})_{k \in \mathbb{N}}$ une suite d'itérés générés par un algorithme convergent donné. On note x^* la limite de la suite et on suppose que : $\forall k \in \mathbb{N}, x^{(k)} \neq x^*$ (sinon l'algorithme convergerait en un nombre fini d'itérations).

La convergence de l'algorithme est dite :

- *Linéaire* si l'erreur $e^{(k)} = \|x^{(k)} - x^*\|$ décroît linéairement c-a-d s'il existe $\tau \in]0; 1[$ tel que

$$\lim_{k \rightarrow +\infty} \frac{\|x^{(k+1)} - x^*\|}{\|x^{(k)} - x^*\|} = \tau.$$

- *Super-linéaire* si

$$\lim_{k \rightarrow +\infty} \frac{\|x^{(k+1)} - x^*\|}{\|x^{(k)} - x^*\|} = 0.$$

- *D'ordre p* s'il existe $\tau \geq 0$ tel que :

$$\lim_{k \rightarrow +\infty} \frac{\|x^{(k+1)} - x^*\|}{\|x^{(k)} - x^*\|^p} = \tau$$

En particulier, si $p = 2$, la convergence est dite quadratique (à partir d'un certain rang, le nombre de chiffres significatifs exacts double à chaque itération).

Bien entendu, on a intérêt à ce que la convergence d'un algorithme soit la plus élevée possible afin de converger vers la solution en un minimum d'itérations pour une précision donnée.

Méthodes de descente

2.1 Méthodes du gradient

La méthode du gradient fait partie de la classe des méthodes dites de descente.

La suite de vecteurs $x^{(k)}$ sera donc générée de la manière suivante :

$$\begin{cases} x^{(0)} & \text{donné dans } \mathbb{R}^n \\ x^{(k+1)} = x^{(k)} + \rho^{(k)} d^{(k)} \end{cases}$$

Avec $d^{(k)} \in \mathbb{R}^n$ et $\rho^{(k)} > 0$.

De nombreux choix existent pour $d^{(k)}$ et $\rho^{(k)}$.

Nous devons en premier lieu choisir la direction de descente.

Rappelons que le développement de Taylor de f au premier ordre au voisinage de $x^{(k+1)}$ est donné par :

$$\begin{aligned} I(x^{(k+1)}) &= f(x^{(k)} + \rho^{(k)} d^{(k)}) \\ &= f(x^{(k)}) + \rho^{(k)} \nabla f(x^{(k)}) d^{(k)} + \rho^{(k)} \|d^{(k)}\| E(x^{(k)}; \rho^{(k)} d^{(k)}) \end{aligned}$$

Or, puisque l'on désire avoir : $f(x^{(k+1)}) \leq f(x^{(k)})$, une solution évidente consiste à prendre : $d^{(k)} = -\nabla f(x^{(k)})$.

Puisqu'alors

$$f(x^{(k+1)}) - f(x^{(k)}) = -\rho^{(k)} \|\nabla f(x^{(k)})\|^2 + o(\rho^{(k)})$$

Nous voyons que si $\rho^{(k)}$ est suffisamment petit, $x^{(k+1)}$ minimisera mieux I que ne le faisait $x^{(k)}$.

La méthode obtenue avec le choix $d^{(k)} = -\nabla f(x^{(k)})$ est appelée méthode du gradient. Lorsque l'on travaille sur une résolution numérique d'un problème, on se donne en général un critère d'arrêt de la forme : $\|x^{(k+1)} - x^{(k)}\| < \varepsilon$.

De plus, puisque la convergence n'est pas toujours assurée, une règle de base est de fixer un nombre maximum d'itérations $k^{\max}$. Sinon l'algorithme génère une suite infinie de $x^{(k)}$.

On obtient alors l'algorithme du Gradient suivant :

Poser $k = 0$

Choisir $x^{(0)}$

Tant que $\|x^{(k+1)} - x^{(k)}\| \geq \varepsilon$ et $k \leq k_{\max}$

Calculer $d^{(k)} = -\nabla f(x^{(k)})$

Calculer $\rho^{(k)}$

Poser $x^{(k+1)} = x^{(k)} + \rho^{(k)} d^{(k)}$

Remarque 2.1. Même si ces méthodes sont conceptuellement très simples et qu'elles peuvent être programmées directement, elles sont souvent lentes dans la pratique.

*Elles convergent mais sous des conditions de convergence souvent complexes.
A titre d'exemple, donnons le résultat suivant.*

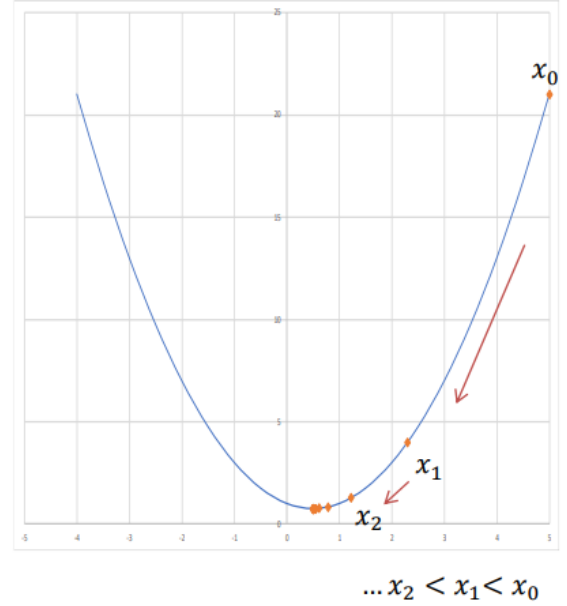
Exemples d'application du gradient de descente

Soit la fonction $f(x) = x^2 - x + 1$. Il n'y a qu'un seul paramètre.
On a :

$$\nabla f(x) = f'(x) = 2x - 1.$$

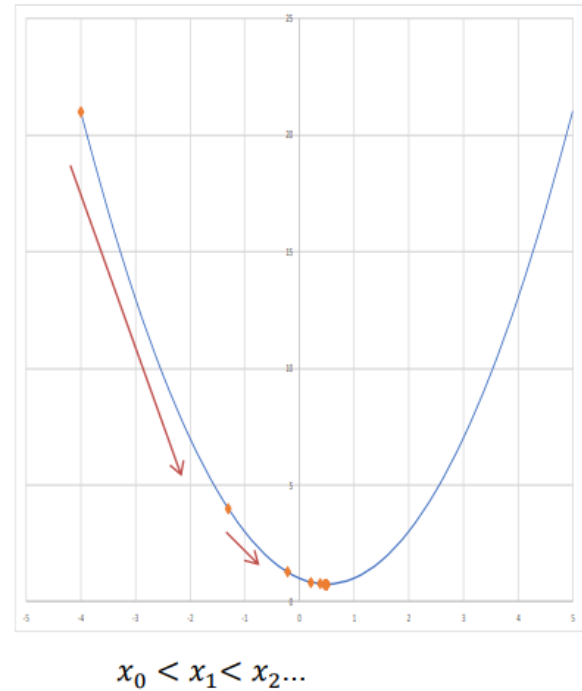
En prenant $x_0 = 5$ et $\rho = 0.3$ on obtient :

i	$x^{(i)}$	$f'(x^{(i)})$	$f(x^{(i)})$
0	5.0000		21.0000
1	2.3000	9.0000	3.9900
2	1.2200	3.6000	1.2684
3	0.7880	1.4400	0.8329
4	0.6152	0.5760	0.7633
5	0.5461	0.2304	0.7521
6	0.5184	0.0922	0.7503
7	0.5074	0.0369	0.7501
8	0.5029	0.0147	0.7500
9	0.5012	0.0059	0.7500
10	0.5005	0.0024	0.7500
11	0.5002	0.0009	0.7500
12	0.5001	0.0004	0.7500
13	0.5000	0.0002	0.7500



En commençant par $x_0 = -4$ on part de l'autre côté et on obtient :

i	$x^{(i)}$	$f'(x^{(i)})$	$f(x^{(i)})$
0	4.0000		21.0000
1	-1.3000	-9.0000	3.9900
2	-0.2200	-3.6000	1.2684
3	0.2120	-1.4400	0.8329
4	0.3848	-0.5760	0.7633
5	0.4539	-0.2304	0.7521
6	0.4816	-0.0922	0.7503
7	0.4926	-0.0369	0.7501
8	0.4971	-0.0147	0.7500
9	0.4988	-0.0059	0.7500
10	0.4995	-0.0024	0.7500
11	0.4998	-0.0009	0.7500
12	0.4999	-0.0004	0.7500
13	0.5000	-0.0002	0.7500



Théorème 2.1. Soit f une fonction de classe C^1 de $\mathbb{R}^n$ dans $\mathbb{R}$, x^* est un minimum de f . Supposons que :

i) f est α -elliptique, c'est-à-dire :

$$\exists \alpha > 0; (\forall (x, y) \in \mathbb{R}^n \times \mathbb{R}^n) \langle (\nabla f(x) - \nabla f(y)), (x - y) \rangle \geq \alpha \|x - y\|^2$$

ii) L'application ∇f est Lipchitzienne, c'est-à-dire :

$$\exists M > 0; (\forall (x, y) \in \mathbb{R}^n \times \mathbb{R}^n) (\|\nabla f(x) - \nabla f(y)\| \leq M \|x - y\|)$$

S'il existe deux réels a et b tels que $\rho^{(k)}$ vérifie $0 < a < \rho^{(k)} < b < \frac{2\alpha}{M^2}$, pour tout $k \geq 0$, alors, la méthode du gradient définie par :

$$x^{(k+1)} = x^{(k)} - \rho^{(k)} \nabla f(x^{(k)})$$

converge pour tout choix de $x^{(0)}$ de façon géométrique, c'est-à-dire :

$$\exists \beta \in]0; 1[, \|x^{(k)} - x^*\| \leq \beta^k \|x^{(0)} - x^*\|.$$

Démonstration. Nous savons que x^* est l'unique solution de l'équation d'Euler :

$$\nabla f(x^*) = 0$$

Notre but est de montrer que $x^{(k)} - x^* \rightarrow 0$ pour $k \rightarrow +\infty$.

Nous avons :

$$x^{(k+1)} - x^* = x^{(k)} - \rho^{(k)} \nabla f(x^{(k)}) - x^* = x^{(k)} - x^* - \rho^{(k)} [\nabla f(x^{(k)}) - \nabla f(x^*)]$$

Utilisons la formule : $\|x - y\|^2 = \|x\|^2 + \|y\|^2 - 2\langle x, y \rangle$ pour tous $x, y \in \mathbb{R}^n$.

Nous avons :

$$\begin{aligned} \|x^{(k+1)} - x^*\|^2 &= \|x^{(k)} - x^*\|^2 + (\rho^{(k)})^2 \|\nabla f(x^{(k)}) - \nabla f(x^*)\|^2 - \\ &\quad 2\rho^{(k)} \langle x^{(k)} - x^*, \nabla f(x^{(k)}) - \nabla f(x^*) \rangle \end{aligned}$$

En utilisant l'ellipticité de f et le fait que ∇f est lipchitzienne, nous obtenons :

$$\|x^{(k+1)} - x^*\|^2 \leq \|x^{(k)} - x^*\|^2 + M^2 (\rho^{(k)})^2 \|x^{(k)} - x^*\|^2 - 2\rho^{(k)} \alpha \|x^{(k)} - x^*\|^2$$

C'est-à-dire :

$$\|x^{(k+1)} - x^*\|^2 \leq \left(1 - 2\alpha\rho^{(k)} + M^2 (\rho^{(k)})^2\right) \|x^{(k)} - x^*\|^2 \quad (2.1)$$

Considérons maintenant la fonction $\varphi : \mathbb{R} \rightarrow \mathbb{R}$ donnée par :

$$\varphi(\rho^{(k)}) = 1 - 2\alpha\rho^{(k)} + M^2 (\rho^{(k)})^2$$

Comme $a \leq \rho^{(k)} \leq b$, il est facile de voir que :

$$\varphi(\rho^{(k)}) \leq \max\{\varphi(a), \varphi(b)\} \quad (2.2)$$

D'autre part, le minimum de φ est atteint en $\frac{\alpha}{M^2}$,
ce qui nous donne :

$$\phi(\rho^{(k)}) \geq 1 - \frac{\alpha^2}{M^2} \quad (2.3)$$

En utilisant l'inégalité de Cauchy Schwartz, nous avons $\forall x, y \in \mathbb{R}^n$

$$\alpha \|x - y\|^2 \leq \langle \nabla f(x) - \nabla f(y), x - y \rangle \leq \|\nabla f(x) - \nabla f(y)\| \|x - y\| \leq M \|x - y\|^2$$

ce qui nous donne, en prenant $x \neq y$:

$$\alpha \leq M \quad (2.4)$$

Comme $\phi(0) = \phi\left(\frac{2\alpha}{M^2}\right) = 1$, on déduit que :

$$\phi(y) \in [0, 1[, \quad y \in \left]0, \frac{2\alpha}{M^2}\right[$$

Posons alors $\beta = \sqrt{\max\{\phi(a), \phi(b)\}}$ Comme $(a, b) \in \left]0, \frac{2\alpha}{M^2}\right[$ on a clairement :

$$0 \leq \beta < 1$$

A l'aide de (2.2) on obtient :

$$\phi(\rho^{(k)}) \leq \beta^2$$

ce qui, avec (2.1) nous donne :

$$\|x^{(k+1)} - x^*\| \leq \beta \|x^{(k)} - x^*\| \quad ; \forall k \in \mathbb{N}$$

Par récurrence, nous déduisons $\forall k \in \mathbb{N}^*$:

$$\|x^{(k)} - x^*\| \leq \beta \|x^{(k-1)} - x^*\| \leq \beta^2 \|x^{(k-2)} - x^*\| \leq \dots \leq \beta^k \|x^{(0)} - x^*\|$$

ce qui achève la preuve du théorème. □

2.1.1 Méthode du gradient à pas fixe

La méthode du gradient à pas fixe définie par :

$$\begin{cases} x^{(0)} \text{ donné dans } \mathbb{R}^n \\ x^{(k+1)} = x^{(k)} - \rho \nabla f(x^{(k)}) \end{cases}$$

C'est une méthode de descente utilisant un pas fixe $\rho^{(k)} = \rho$ (indépendamment de k) et la stratégie de Cauchy (i.e $d^{(k)} = -\nabla f(x^{(k)})$), pour le choix de la direction de descente.

La convergence de la méthode est assurée sous les hypothèses du théorème précédent.

En particulier pour un choix de pas ρ vérifiant :

$$0 < \rho < \frac{2\alpha}{M^2}$$

C'est la méthode de gradient la plus simple à utiliser.

Remarque 2.2.

- En général il est difficile de connaître α et M (en supposant qu'ils existent). Alors dans la pratique on prend un pas ρ assez petit pour être sûr d'avoir la convergence. Mais dans ce cas la convergence peut être lente !
- Dans le cas quadratique, α est connu (voir [6, p.9]).

2.1.2 Méthode du gradient à pas optimal

• Cas d'une fonction f quelconque.

La méthode du gradient à pas optimal consiste à choisir $\rho^{(k)}$ comme étant le minimum de la fonction $q(\rho) = f(x^{(k)} - \rho \nabla f(x^{(k)}))$.

Dans ce cas, nous avons le même résultat de convergence que précédemment sous des hypothèses faibles sur f .

Nous avons donc le résultat suivant.

Théorème 2.2. *Soit $f : \mathbb{R}^n \rightarrow \mathbb{R}$ une fonction α -elliptique et M -lipschitzienne.*

Alors la méthode du gradient à pas optimal donnée par :

$$\begin{cases} x^{(0)} \text{ donné dans } \mathbb{R}^n \\ x^{(k+1)} = x^{(k)} - \rho^{(k)} \nabla f(x^{(k)}) \end{cases},$$

avec $\rho^{(k)}$ définie par :

$$\rho^{(k)} = \min_{\rho \in \mathbb{R}} f(x^{(k)} - \rho \nabla f(x^{(k)})),$$

est bien définie et converge vers x^ la solution du problème d'optimisation.*

• Cas d'une fonction f quadratique.

On suppose ici que la fonction f associée à une matrice A symétrique définie positive (SDP) est quadratique. Nous devons calculer $\rho^{(k)} \in \mathbb{R}$ qui minimise la fonction $q : \mathbb{R} \rightarrow \mathbb{R}$ donnée par : $q(\rho) = f(x^{(k)} - \rho \nabla f(x^{(k)}))$.

Alors $\rho^{(k)}$ satisfait nécessairement $q'(\rho^{(k)}) = 0$.

Un calcul simple nous donne :

$$q'(\rho) = -\langle \nabla f(x^{(k)} - \rho \nabla f(x^{(k)})), \nabla f(x^{(k)}) \rangle;$$

c'est-à-dire, comme : $\nabla f(x^{(k)}) = Ax^{(k)} - b$.

$$\begin{aligned} q'(\rho) &= -\langle A(x^{(k)} - \rho \nabla f(x^{(k)})), Ax^{(k)} - b \rangle \\ &= -\|Ax^{(k)} - b\|^2 + \rho \langle A(Ax^{(k)} - b), Ax^{(k)} - b \rangle. \end{aligned}$$

On obtient alors,

$$\rho^{(k)} = \frac{\|Ax^{(k)} - b\|^2}{\langle A(Ax^{(k)} - b), Ax^{(k)} - b \rangle}.$$

Remarquons qu'on a,

$$\langle A(Ax^{(k)} - b), Ax^{(k)} - b \rangle > 0 \text{ (car } A \text{ est SDP et } Ax^{(k)} - b = \nabla f(x^{(k)}) \neq 0 \text{)}.$$

Donc la méthode du gradient à pas optimal dans le cas f quadratique est donnée par :

$$x^{(k+1)} = x^{(k)} - \rho^{(k)}(Ax^{(k)} - b).$$

Avec $\rho^{(k)}$ donnée par la formule précédente (valable uniquement pour $Ax^{(k)} - b \neq 0$).

Comme $q'(\rho) = 0$, on déduit immédiatement que les directions successives de descente sont orthogonales, c'est-à-dire :

$$\langle \nabla f(x^{(k+1)}), \nabla f(x^{(k)}) \rangle = 0.$$

Remarque 2.3. *Même pour le gradient à pas optimal qui est en principe la meilleur de ces méthodes d'un point de vu de rapidité de convergence, celle-ci peut être lente car elle est altérée par un mauvais conditionnement de la matrice Hessienne H de f .*

Par ailleurs, on peut considérer des critères de convergence sur le gradient de f en $x^{(k)}$: $\|\nabla f(x^{(k)})\| < \varepsilon$.

2.1.3 Méthode du gradient conjugué

• Cas quadratique :

Considérons la forme quadratique suivante :

$$f(x) = \frac{1}{2} \langle Ax, x \rangle - \langle b, x \rangle,$$

avec $A \in M_n(\mathbb{R})$ une matrice symétrique définie positive, $b \in \mathbb{R}^n$.

On considère dans ce paragraphe le problème de minimisation sans contrainte suivant :

$$\min_{x \in \mathbb{R}^n} f(x) = \min_{x \in \mathbb{R}^n} \frac{1}{2} \langle Ax, x \rangle - \langle b, x \rangle.$$

Dans cette méthode, on démarre d'un point $x^{(0)} \in \mathbb{R}^n$, $d^{(0)} = -r^{(0)} = -\nabla f(x^{(0)}) = b - Ax^{(0)}$. Les directions $d^{(1)}, \dots, d^{(n-1)}$ sont calculées à chaque itération.

A l'itération k on a :

$$d^{(k)} = -r^{(k)} + \beta^{(k-1)} d^{(k-1)},$$

où $\beta^{(k-1)}$ est obtenu de sorte que $d^{(k)}$ soit A -conjugué avec les autres vecteurs $d^{(0)}, \dots, d^{(k-1)}$. En d'autres termes, on doit avoir :

$$\langle Ad^{(k)}, d^{(i)} \rangle = 0, \quad i = 0, \dots, k-1.$$

Dans l'appellation gradient conjugué on trouve les deux mots : gradient et conjugué.

- Le mot *gradient* est utilisé car $d^{(k)}$ est calculé à partir du gradient au point $x^{(k)}$.
- Le mot *conjugué* est aussi justifié car les directions $d^{(0)}, \dots, d^{(n-1)}$ sont A -conjuguées.

Principe de la méthode

On démarre d'un point quelconque $x^{(0)} \in \mathbb{R}^n$.

Le principe de la méthode consiste à partir d'un point $x^{(0)}$ à minimiser $f(x)$ successivement suivant n directions linéairement indépendantes $d^{(0)}, \dots, d^{(n-1)}$ possédant la propriété d'être mutuellement conjuguées par rapport à la forme quadratique f .

Pour $k = 0$,

Calculer $d^{(0)} = -r^{(0)} = b - Ax^{(0)}$,

$$\rho^{(0)} = -\frac{\langle r^{(0)}, d^{(0)} \rangle}{\langle Ad^{(0)}, d^{(0)} \rangle}$$

Supposons qu'à l'itération k on ait : $x^{(k)}$ et $d^{(k)}$

Ceci nous permettra de calculer :

$$\begin{aligned} r^{(k)} &= Ax^{(k)} - b, \\ \rho^{(k)} &= -\frac{\langle r^{(k)}, d^{(k)} \rangle}{\langle Ad^{(k)}, d^{(k)} \rangle}, \\ x^{(k+1)} &= x^{(k)} + \rho^{(k)} d^{(k)}, \\ r^{(k+1)} &= Ax^{(k+1)} - b, \\ d^{(k+1)} &= -r^{(k+1)} + \beta^{(k)} d^{(k)}. \end{aligned}$$

$\beta^{(k)}$ est choisi de sorte que :

$$\langle Ad^{(k+1)}, d^{(k)} \rangle = 0,$$

Puisque $d^{(k+1)} = -r^{(k+1)} + \beta^{(k)}d^{(k)}$, ce qui nous donne :

$$\langle A(-r^{(k+1)} + \beta^{(k)}d^{(k)}), d^{(k)} \rangle = 0,$$

ou encore :

$$\langle A(\beta^{(k)}d^{(k)}), d^{(k)} \rangle = \langle Ar^{(k+1)}, d^{(k)} \rangle,$$

et finalement

$$\beta^{(k)} = \frac{\langle Ar^{(k+1)}, d^{(k)} \rangle}{\langle Ad^{(k)}, d^{(k)} \rangle}$$

En résumé, on aura l'algorithme suivant :

1. Choisir $x^{(0)} \in \mathbb{R}^n$.
2. Calculer $r^{(0)} = Ax^{(0)} - b$. Si $r^{(0)} = 0$ **stop**. Sinon poser : $d^{(0)} = -r^{(0)}$. Poser $k = 0$.
3. Tant que $\|x^{(k+1)} - x^{(k)}\| \geq \varepsilon$ et $k \leq k_{\max}$. Faire

— Calculer :

$$\rho^{(k)} = -\frac{\langle r^{(k)}, d^{(k)} \rangle}{\langle Ad^{(k)}, d^{(k)} \rangle}.$$

— Calculer :

$$x^{(k+1)} = x^{(k)} + \rho^{(k)}d^{(k)}$$

— Calculer :

$$r^{(k+1)} = Ax^{(k+1)} - b. \text{ Si } r^{(k+1)} = 0 \text{ stop,}$$

— Calculer :

$$\beta^{(k)} = \frac{\langle Ar^{(k+1)}, d^{(k)} \rangle}{\langle Ad^{(k)}, d^{(k)} \rangle}$$

— Calculer :

$$d^{(k+1)} = -r^{(k+1)} + \beta^{(k)}d^{(k)}.$$

— Poser $k = k + 1$ et retourner à 3.

Fin

Théorème 2.3. (Convergence de la méthode du gradient conjugué quadratique)

Partant d'un point initial $x^{(0)} \in \mathbb{R}^n$ quelconque, l'algorithme de gradient conjugué converge vers la solution optimale unique x^* du problème dans n itérations, ie qu'on a : $x^{(n)} = x^*$ et $Ax^n = Ax^* = b$.

• **Cas non quadratique avec la variance de :**

La méthode de *Flétcher-Reeves*(1964) est une extension directe de la méthode précédente au cas des fonctions quelconques.

Cette méthode est très intéressante, d'une part parce qu'elle nécessite le stockage de très peu d'informations (essentiellement trois vecteurs de dimension n) ; d'autre part, par sa vitesse de convergence très supérieure à celle des algorithmes du gradient classique.

Partir de $k = 0$.

1. Choisir $x^{(0)} \in \mathbb{R}^n$ quelconque.
- Choisir $\varepsilon > 0$.
- Choisir $\varepsilon_1 > 0$.

2. Pour $k = 0$, calculer $r^{(0)} = \nabla f(x(0))$.
Poser $d^{(0)} = -r^{(0)}$.
3. Tant que $\|x^{(k+1)} - x^{(k)}\| \geq \varepsilon$ et $k \leq k_{\max}$ faire :
Si $\|r^{(k)}\| < \varepsilon_1$ alors arrêt : $x^* = x^{(k)}$. Sinon
4. Calculer :
$$\beta^{(k)} = \frac{\|\nabla f(x^{(k)})\|^2}{\|\nabla f(x^{(k-1)})\|^2}$$
5. Calculer $d^{(k+1)} = -r^{(k+1)} + \beta^{(k)}d^{(k)}$
6. Calculer $\rho^{(k)}$ approchant le minimum de la fonction $\rho \mapsto f(x^{(k)} + \rho d^{(k)})$ en utilisant une recherche linéaire du pas de descente.
7. Calculer $x^{(k+1)} = x^{(k)} + \rho^{(k)}d^{(k)}$
8. Poser $k = k + 1$
Retourner à l'étape 4
Fin

Remarque 2.4. On voit que l'on garde en mémoire à chaque étape, que les gradients en $x^{(k)}$ et $x^{(k+1)}$ et la direction de déplacement courante $d^{(k)}$.

Il est important de remarquer que la convergence globale de la mémoire de Fletcher-Reeves n'est assurée que si l'on procède à une réinitialisation périodique.

Par exemple, toute les n itérations, on repartira du dernier point obtenu avec, comme direction de déplacement, le gradient en ce point.

La convergence globale de cette procédure découle alors de la convergence des méthodes de gradient à pas optimal.

2.1.4 Méthode du gradient stochastique

L'algorithme du gradient stochastique est une méthode de descente de gradient (itérative) utilisée pour la minimisation d'une fonction objectif qui est écrite comme une somme de fonctions différentiables.

L'apprentissage automatique s'intéresse au problème de la minimisation d'une fonction objectif qui a la forme d'une somme :

$$Q(\omega) = \frac{1}{n} \sum_{i=1}^n Q_i(\omega),$$

où le paramètre ω^* qui minimise $Q(\omega)$ doit être estimé. Chacune des fonctions Q_i est généralement associée avec la i -ème observation de l'ensemble des données (utilisées pour l'apprentissage).

La plupart du temps on cherche à trouver la valeur de ω minimisant le risque empirique :

$$Q(\omega) = \frac{1}{n} \sum_{i=1}^n Q_i(\omega).$$

Une méthode de descente de gradient est le plus souvent employée. Il s'agit d'une méthode itérative construisant une suite $(\omega)_n$ définie par :

$$\omega_{n+1} = \omega_n - \rho \nabla Q(\omega) = \omega_n - \rho \frac{1}{n} \sum_{i=1}^n \nabla Q_i(\omega)$$

ρ est le pas d'iteration. On l'appelle fréquemment Taux d'apprentissage.

En apprentissage, le nombre n de données utilisées peut être très grand (des millions de données ...). Dans ce cas le calcul de la somme des gradients peut prendre un temps prohibitif.

Dans la méthode de descente du gradient stochastique (SGD), on procède à une simplification drastique. Au lieu de calculer le gradient du risque empirique ∇Q exactement, chaque iteration estime ce gradient sur la base d'une seule mesure. L'algorithme réalise ainsi une mise à jour après chaque exemple.

- choix de ω_0 et de ρ
- Pour i allant de 1 à n , faire
choisir aléatoirement un exemple noté $l(i)$

$$\omega_{n+1} = \omega_n - \rho \nabla Q(\omega) = \omega_n - \rho \frac{1}{n} \sum_{i=1}^n Q_{l(i)}(\omega)$$
- fin

2.2 Méthode de Newton

La méthode de Newton est une méthode de recherche de zéros d'une fonction $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$. L'idée de cette méthode consiste dans notre cas de résoudre le système d'équations : $\nabla f(x) = 0$ avec $\nabla f : \mathbb{R}^n \rightarrow \mathbb{R}^n$.

Soit $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ une fonction de classe C^1 .

On suppose que l'équation $f(x) = 0$ possède au moins une solution notée x^* et que la matrice $Df(x^{(k)})$ est une matrice inversible.

L'algorithme de Newton peut s'écrire sous la forme présentée dans la table ci-dessous.

Poser $k = 0$
 Choisir $x^{(0)}$ dans un voisinage de x^*
 Choisir $\varepsilon > 0$
 Tant que $\|x^{(k+1)} - x^{(k)}\| \geq \varepsilon$ et $k \leq k_{max}$ faire
 Résoudre le système linéaire $[Df(x^{(k)})]d^{(k)} = -f(x^{(k)})$
 Poser $x^{(k+1)} = x^{(k)} + d^{(k)}$
 Poser $k = k + 1$
 Fin tant que

Remarque 2.5. : Un des gros problèmes de cette méthode est que le choix de $x^{(0)}$ joue un grand rôle sur la convergence ou non de la méthode de Newton.

La méthode est très sensible à l'initialisation.

Il peut aussi arriver que la méthode converge vers un extremum qui n'est pas celui recherché.

Une approche possible consiste à faire tourner quelques itérations d'une méthode de gradient pour approcher x^* et de considérer l'itéré résultant comme le point de départ de la méthode de Newton.

L'avantage de la méthode de Newton est sa grande rapidité de convergence : la convergence de la méthode de Newton est quadratique.

Applications : Classification supervisée

3.1 Définition

La classification est une technique qui permet de classer les données dans un nombre donné de classes, elle peut être effectuée sur des données structurées ou non structurées. L'objectif principal d'un problème de classification est d'identifier la catégorie/classe à laquelle une nouvelle donnée appartient.

3.2 Exemples

- Classification des pourriels
- Prédiction de la volonté de remboursement des prêts des clients des banques
- Identification des cellules tumorales du cancer
- Analyse des sentiments : Déterminer si un texte a un sentiment positif ou négatif.
- Classification des médicaments
- Détection des points clés du visage
- Détection des piétons dans une conduite automobile
- Détection de spam : Identifier les e-mails indésirables
- Reconnaissance d'image : Catégoriser les images en différentes classes.

3.3 Algorithmes de classification

1. Arbre de décision

Un arbre décisionnel est un outil de modélisation prédictive qui peut être utilisé aussi bien pour la régression que pour la classification. Dans le cas d'un arbre décisionnel de classification, la variable de décision est une catégorie, alors que dans le cas d'un arbre décisionnel de régression, la variable de décision est continue. Les arbres de décision sont probablement l'un des algorithmes de machine learning les plus compréhensibles et peuvent être interprétés de manière relativement simple. Ces arbres peuvent être construits à travers une approche algorithmique qui sépare les données dépendant de différentes conditions. Dans un arbre, les feuilles (ou nodes) représentent les valeurs de la variable-cible et les

embranchements correspondent à une combinaison de variables d'entrée menant à ces valeurs.

La création d'un arbre décisionnel nécessite les étapes suivantes :

- Choisir une variable de prédiction qui classe les données de la meilleure manière et qui attribue chaque point à la première feuille.
- Descendre le long de l'arbre, en partant de la première feuille, tout en prenant des décisions par rapport à la classification de chaque point
- Retourner à l'étape 1 et répéter ce même processus jusqu'à ce que chaque point soit classifié.

2. La méthode des K plus proches voisins

La méthode des K plus proches voisins est l'un des algorithmes d'apprentissage supervisé, qui peut être utilisé pour un problème de régression et de classification. Cependant, elle reste principalement utilisée pour les problèmes de classification. Cet algorithme s'entraîne sur des données étiquetées et tente de prédire la classe associée à des variables d'essai, en calculant la distance entre la variable d'essai explicative et les points d'entraînement. En d'autres termes, cette méthode enregistre les points d'entraînement puis classe les nouveaux points dépendant de leur similarité avec les données enregistrées.

L'algorithme pour les modèles des K plus proches voisins est le suivant :

- Choisir le nombre K de voisins
- Calculer la distance Euclidienne des K voisins
- Prendre les K plus proches voisins, dépendant du résultat du calcul de la distance
- Parmi ce K voisin, compter le nombre de points dans chaque classe
- Attribuer le nouveau point à la classe dont le nombre de voisins est maximal
- Le modèle est terminé

3. Forêt d'arbres décisionnels

Nous venons de comprendre le fonctionnement d'un arbre décisionnel. Nous pouvons maintenant examiner l'algorithme de forêt d'arbres décisionnels. En effet, une forêt d'arbres décisionnels est un algorithme d'apprentissage supervisé, qui peut être utilisé pour la régression et la classification. Ce genre d'algorithme peut créer un ensemble d'arbres décisionnels aléatoires, à partir d'un groupe de données. L'algorithme identifie la meilleure prédiction au sein de chaque arbre et sélectionne la meilleure décision par le biais d'un vote. Ainsi, cela nécessite plusieurs arbres produisant des valeurs qui réduisent le surapprentissage de l'algorithme en faisant une moyenne des résultats.

4. La régression logistique

La régression logistique est une technique de classification linéaire, qui est utilisée afin de prédire la probabilité d'une variable-cible et d'établir une relation linéaire entre la variable d'entrée et celle de sortie. De manière générale, la variable explicative est binaire et est donc représentée par 1, lors d'un succès/oui, ou par un 0, lors d'un échec/non. Cependant, il peut y avoir plus de deux catégories de variable explicatives : cela nécessite alors l'utilisation d'une régression logistique multiple.

i) Régression logistique binaire

La régression logistique binaire constitue la forme de régression logistique la plus répandue et utilisée. Elle implique une variable explicative prenant seulement deux valeurs : 1 ou 0.

$$S(X) = \theta_0 + x_1\theta_1 + x_2\theta_2 + x_3\theta_3 + \dots + x_n\theta_n$$

Si on attribue la valeur $x_0 = 1$, alors la formule précédente devient :

$$S(X) = \sum_0^{n+1} \theta_i x_i$$

Dans ce cas, X correspond au vecteur de $(x_1, x_2, \dots, x_n)$. De la même manière, si on écrit ϑ comme étant le vecteur $(\theta_1, \theta_2, \dots, \theta_n)$ alors on peut réécrire la formule précédente comme : $S(X) = \vartheta X$.

Cette fonction d'hypothèse est appelée la fonction score. Idéalement, on souhaite que $(\theta_1, \theta_2, \dots, \theta_n)$ soit tel que :

* $S(X) > 0$ lorsque la variable explicative est 1

* $S(X) < 0$ lorsque la variable explicative 0

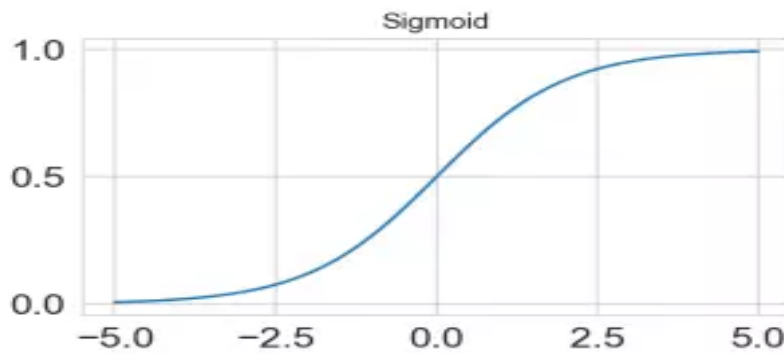
Dès lors, en utilisant la fonction score, on peut calculer la fonction sigmoïde, ou courbe en S, qui est utilisée pour les modèles de régression logistique. Cette fonction rend des valeurs entre 0 et 1 et ce résultat peut être interprété comme la probabilité qu'une observation X corresponde à la valeur à expliquer de 1. Lorsqu'on applique la fonction sigmoïde sur notre fonction score, on obtient une fonction d'hypothèse, pour notre modèle, de la forme suivante :

$$H(X) = \text{Sigmoid}(S(X)) = \frac{1}{1 + e^{-\vartheta X}} = P(y = 1 | X; \vartheta)$$

Ainsi, cela correspond à la probabilité qu'une observation X soit de classe 1, avec pour paramètre ϑ . De manière équivalente, la probabilité que X soit de classe 0 est donnée par :

$$P(y = 0 | X; \vartheta) = 1 - P(y = 1 | X; \vartheta)$$

Le graphe suivant représente la fonction sigmoïde précédente :



ii) Régression logistique multiple

Dans ce type de classification, la variable explicative peut prendre trois ou plus valeurs possibles. Par exemple, on peut avoir une variable de Type A, de Type B ou de Type C. La variable explicative serait de forme non-ordonnée, ce qui implique que les valeurs n'auraient pas de signification quantitative. La formule précédente, présentée dans la régression logistique binaire, peut être adaptée pour la version multinomiale.

3.4 Quelques problèmes de classification supervisée

3.4.1 Classification d'images

La classification d'images consiste à assigner une étiquette de classe prédéfinie à une image donnée. Par exemple, dans un jeu de données de reconnaissance d'objets, chaque image pourrait être classée comme appartenant à l'une des catégories suivantes : chien, chat, voiture, avion, etc.

Exemple : Supposons que nous ayons un ensemble de données contenant des milliers d'images de chiens et de chats. Chaque image est associée à une étiquette indiquant si l'image représente un chien ou un chat. L'objectif est de construire un modèle de classification qui, à partir de nouvelles images non étiquetées, puisse prédire correctement si chaque nouvelle image représente un chien ou un chat.

La classification d'images est utilisée dans :

- Reconnaissance faciale
- Analyse de vidéos de surveillance
- Systèmes de diagnostic médical par imagerie (par exemple, identification de tumeurs dans des images de radiologie)

3.4.2 Classification de texte

La classification de texte consiste à attribuer une ou plusieurs catégories prédéfinies à un document ou un texte donné. Ce problème est omniprésent dans le traitement automatique du langage naturel (TALN) et trouve de nombreuses applications dans l'industrie.

Exemple : Un exemple classique est la classification des e-mails en spam ou non-spam. Dans ce cas, nous avons un ensemble de données d'e-mails étiquetés, certains étant marqués comme spam et d'autres comme non-spam. L'objectif est de construire un modèle de classification qui peut analyser le contenu de nouveaux e-mails et prédire s'ils sont des spams ou non.

Applications :

- Filtrage des spams dans les systèmes de messagerie électronique (SaneBox et CleanEmail sont deux exemples de logiciels utilisés)
- Analyse de sentiments dans les avis des clients (par exemple, déterminer si un avis est positif, négatif ou neutre)
- Classification des articles de presse par catégorie (politique, sport, technologie, éducation, etc.)

3.5 Une illustration

Ici nous allons appliquer la méthode de descente à un problème de classification supervisée.

Problème de Classification Supervisée : Prédire la Volonté de Remboursement des Prêts des Clients des Banques

Description du Problème

Les banques et les institutions financières souhaitent prédire la probabilité qu'un client rembourse son prêt à temps. Ce problème est crucial pour gérer les risques de crédit et prendre des décisions éclairées sur l'octroi de prêts. Nous avons un ensemble de données contenant des

informations sur les clients et leur historique de crédit. L'objectif est de construire un modèle de classification pour prédire si un client remboursera son prêt à temps ou non.

Étapes pour Résoudre le Problème

1. Collecte des Données :
 - Recueillir des données sur les clients, y compris les caractéristiques démographiques, les informations financières, et l'historique de crédit.
 - Les caractéristiques peuvent inclure l'âge, le revenu, la durée de l'emploi, le montant du prêt, le ratio dette/revenu, le nombre de défauts de paiement antérieurs, etc.
 - L'étiquette est binaire : 1 (remboursera à temps) ou 0 (ne remboursera pas à temps).
2. Prétraitement des Données :
 - Nettoyage des Données : Traiter les valeurs manquantes.
 - Encodage des Variables Catégorielles : Convertir les variables catégorielles en variables numériques (par exemple, en utilisant l'encodage one-hot).
 - Normalisation : Mettre à l'échelle les variables numériques pour faciliter l'entraînement du modèle.
3. Division du Jeu de Données :
 - Diviser les données en ensembles d'entraînement et de test pour évaluer la performance du modèle.
4. Sélection du Modèle :
 - Choisir un modèle de classification approprié, comme la régression logistique, les arbres de décision, les forêts aléatoires, ou les méthodes de boosting.

Modèle de Classification : Régression Logistique

Nous utiliserons la régression logistique, un modèle de classification binaire simple mais efficace, pour ce problème.

Fonction de Coût et Optimisation

La fonction de coût pour la régression logistique est la fonction de log-vraisemblance négative :

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))]$$

où m est le nombre d'exemples dans l'ensemble de données, $y^{(i)}$ est la véritable étiquette de l'exemple i , $x^{(i)}$ est le vecteur de caractéristiques de l'exemple i et $h_{\theta}(x)$ est la sortie du modèle (probabilité prédite) pour les caractéristiques x , donnée par la fonction sigmoïde :

$$h_{\theta}(x) = \sigma(\theta^T x + b) = \frac{1}{1 + e^{-(\theta^T x + b)}}$$

Descente de Gradient

Pour minimiser la fonction de coût, nous utilisons la descente de gradient. Les mises à jour des paramètres se font selon la règle suivante :

$$\theta_j := \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j}$$

avec α le taux d'apprentissage.

Implémentation en Python

Voici une implémentation en Python de la régression logistique pour prédire la volonté de remboursement des prêts :

```
import numpy as np
import pandas as pd
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Générer des données synthétiques
X, y = make_classification(n_samples=1000, n_features=10, n_informative=5,
n_redundant=2, n_clusters_per_class=1, random_state=42)

# Créer un DataFrame pandas pour les caractéristiques et les étiquettes
data = pd.DataFrame(X, columns=[f'feature_{i}' for i in range(1, 11)])
data['label'] = y

# Afficher les premières lignes des données
print(data.head())

# Séparer les caractéristiques des étiquettes
X = data.drop('label', axis=1)
y = data['label']
# Diviser les données en ensembles d'entraînement et de test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Normalisation des caractéristiques
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Construction et Entraînement du Modèle
model = LogisticRegression()
model.fit(X_train, y_train)

# Prédications et Évaluation
y_pred = model.predict(X_test)

# Calculer la précision du modèle
accuracy = accuracy_score(y_test, y_pred)
print(f'Précision du modèle : {accuracy:.2f}')

# Calculer et afficher la matrice de confusion
conf_matrix = confusion_matrix(y_test, y_pred)
print('Matrice de confusion :')
print(conf_matrix)
```

```
# Afficher le rapport de classification
class_report = classification_report(y_test, y_pred)
print('Rapport de classification :')
print(class_report)
```

Explication des Étapes du Code

1. Prétraitement des Données :
 - Les caractéristiques des clients sont séparées de l'étiquette.
 - Les données sont divisées en ensembles d'entraînement et de test.
 - Les caractéristiques sont normalisées pour une meilleure performance du modèle.
2. Construction et Entraînement du Modèle :
 - Un modèle de régression logistique est construit et entraîné sur les données d'entraînement.
3. Prédictions et Évaluation :
 - (a) Les prédictions sont effectuées sur l'ensemble de test.
 - (b) La précision du modèle, la matrice de confusion et le rapport de classification sont calculés et affichés pour évaluer la performance du modèle.

Conclusion

Ce processus de classification supervisée permet de construire un modèle capable de prédire la volonté de remboursement des prêts des clients des banques. Les étapes incluent la préparation des données, la sélection et l'entraînement du modèle, et l'évaluation des performances. Cette approche aide les institutions financières à gérer les risques de crédit de manière plus efficace en identifiant les clients susceptibles de rembourser leurs prêts à temps.

3.6 Analyse des résultats

feature_1	feature_2	feature_3	feature_4	feature_5
2.391920	0.443436	1.556545	-0.767381	1.148846
1.607854	1.475931	0.446662	0.902533	1.905526
-0.895996	1.659461	0.688592	-1.359696	1.445481
1.587328	-0.047797	-1.732550	0.957243	0.839766
0.592534	0.436035	0.526122	-0.117240	1.580351

feature_6	feature_7	feature_8	feature_9	feature_10	label
1.492141	1.611632	-0.834854	-1.688854	-0.077773	0
-0.813787	1.259474	1.435686	1.178518	-1.131406	0
1.207164	0.061018	-1.951932	-1.829532	1.277233	0
1.622668	-1.082722	-2.340958	-2.619970	-0.588287	1
-0.288508	1.925248	-0.274555	-0.526378	0.733448	1

1. Précision du modèle : 0.98

```
accuracy = accuracy_score(y_test, y_pred)
print(f'Précision du modèle : {accuracy:.2f}')
```

La précision du modèle est une mesure de la proportion des prédictions correctes (tant les vrais positifs que les vrais négatifs) parmi le nombre total de prédictions. La précision est calculée comme suit :

$$\text{Précision} = \frac{\text{Nombre de prédictions correctes}}{\text{Nombre total de prédictions}}$$

La sortie ici est 0.98 ce qui signifie que le modèle a correctement prédit 98% des échantillons de l'ensemble test.

2. Matrice de Confusion :

```
conf_matrix = confusion_matrix(y_test, y_pred)
print('Matrice de confusion :')
print(conf_matrix)
```

La matrice de confusion est un tableau 2×2 pour les problèmes de classification binaire, où : la première ligne représente les vrais positifs (VP) et les faux négatifs (FN) et la deuxième ligne représente les faux positifs (FP) et les vrais négatifs (VN).

98	0
4	98

TABLE 3.1 – Matrice de Confusion

- 98 vrais positifs (clients qui remboursent leur prêt à temps et correctement prédit).
- 0 faux négatifs (clients qui remboursent leur prêt à temps mais incorrectement prédit comme ne remboursant pas à temps).
- 4 faux positifs (clients qui ne remboursent pas leur prêt à temps mais incorrectement prédit comme remboursant à temps).
- 98 vrais négatifs (clients qui ne remboursent pas leur prêt à temps et correctement prédit).

3. Rapport de classification

```
class_report = classification_report(y_test, y_pred)
print('Rapport de classification :')
print(class_report)
```

Le rapport de classification donne des informations détaillées sur la performance du modèle, y compris la précision, le rappel, et le score F1 pour chaque classe.

- Précision : La proportion de prédictions positives correctes par rapport au total des prédictions positives.

$$\text{Précision} = \frac{VP}{VP + FP}$$

- Rappel : La proportion de vrais positifs corrects par rapport au total des réels positifs.

$$\text{Rappel} = \frac{VP}{VP + FN}$$

	precision	recall	f1-score	support
Classe 0	0.96	1.00	0.98	98
Classe 1	1.00	0.96	0.98	102
exactitude	0.98			200
moyenne macro	0.98	0.98	0.98	200
moyenne pondéré	0.98	0.98	0.98	200

TABLE 3.2 – Rapport de Classification

- Score F1 : La moyenne harmonique de la précision et du rappel.

$$\text{Score F1} = 2 \cdot \frac{\text{Précision} \cdot \text{Rappel}}{\text{Précision} + \text{Rappel}}$$

- Support : Le nombre d'occurrences réelles de la classe dans l'ensemble de test.

Interprétation

- **Précision Globale** : La précision globale de 0.98 (ou 87%) montre que le modèle a globalement de bonnes performances. Cependant, la précision seule ne suffit pas pour évaluer la qualité d'un modèle, surtout dans le cas d'un déséquilibre de classes.
- **Matrice de Confusion** : La matrice de confusion montre que le modèle fait relativement peu d'erreurs, mais il y a encore des faux positifs et des faux négatifs, ce qui peut avoir des implications différentes selon le contexte de l'application (par exemple, refuser un prêt à un client solvable peut avoir des conséquences différentes que d'approuver un prêt à un client non solvable).
- **Rapport de Classification** : Le modèle a une bonne précision et un bon rappel pour les deux classes, avec des scores F1 équilibrés. Cela montre que le modèle est performant à la fois pour identifier les clients qui remboursent à temps et ceux qui ne remboursent pas à temps.

Ces résultats suggèrent que le modèle est fiable, mais il pourrait être amélioré avec des données supplémentaires, une meilleure sélection des caractéristiques, ou en essayant d'autres algorithmes de classification.

Conclusion

En conclusion, les méthodes de descente, comme la descente de gradient, sont essentielles en optimisation et en apprentissage automatique, particulièrement pour la classification supervisée. Elles ajustent les paramètres des modèles pour minimiser les fonctions de coût, améliorant ainsi la précision des prédictions.

Ces méthodes sont cruciales dans des modèles tels que la régression logistique, les réseaux de neurones et les machines à vecteurs de support. Malgré les défis comme le choix du taux d'apprentissage et les minima locaux, des techniques avancées comme la descente de gradient stochastique et Adam aident à surmonter ces problèmes.

Maîtriser ces techniques est indispensable pour développer des modèles efficaces, et les progrès futurs dans ce domaine continueront à renforcer les capacités de l'intelligence artificielle et de l'apprentissage automatique.

Bibliographie

- [1] SOULAMI Hanae, Mémoire de fin d'études, *Méthode de descente en Optimisation Numérique* 9 Juin 2016.
- [2] BIERLAIRE Michel, *Exposé sur les méthodes de descentes* EPFL - Laboratoire Transport et Mobilité - ENAC
- [3] ASSASSI Houda , *La descente du gradient stochastique parallélisé avec Open MP dans un contexte de l'intelligence artificielle*, Mémoire de projet de fin d'études, (2021).
- [4] ELADASSI Imen, *Sur la convergence de quelques méthodes hybrides du gradient conjugué non linéaire*, Mémoire de master académique en mathématiques (2021).
- [5] TOKPLO Christelle *Prédiction du credit scoring par l'algorithme des réseaux de neurones* Mémoire de master 2 en Recherche Opérationnelle IMSP 2020.
- [6] Luca Amodei Université de Toulouse, Master1, UE OPTIMISATION Support de cours. 15 décembre 2017